МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 2

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил: доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы:

Научиться переводить asm код на язык C.

Задания на лабораторную работу:

Написать код на C из ассемблерного кода.

1. Теоретические сведения

Подробно разберем перевод на С на примере.

```
0: 0c 94 34 00 jmp 0x68 ; 0x68
4: 0c 94 3e 00 jmp 0x7c ; 0x7c
8: 0c 94 3e 00 jmp 0x7c ; 0x7c
c: 0c 94 3e 00 jmp 0x7c ; 0x7c
10: 0c 94 3e 00 jmp 0x7c ; 0x7c
14: 0c 94 3e 00 jmp 0x7c ; 0x7c
18: 0c 94 3e 00 jmp 0x7c ; 0x7c
1c: 0c 94 3e 00 jmp 0x7c ; 0x7c
20: 0c 94 3e 00 jmp 0x7c ; 0x7c
24: 0c 94 3e 00 jmp 0x7c ; 0x7c
28: 0c 94 3e 00 jmp 0x7c ; 0x7c
2c: 0c 94 3e 00 jmp 0x7c ; 0x7c
30: 0c 94 3e 00 jmp 0x7c ; 0x7c
34: 0c 94 3e 00 jmp 0x7c ; 0x7c
38: 0c 94 3e 00 jmp 0x7c ; 0x7c
3c: 0c 94 3e 00 jmp 0x7c ; 0x7c
40: 0c 94 3e 00 jmp 0x7c ; 0x7c
44: 0c 94 3e 00 jmp 0x7c ; 0x7c
48: 0c 94 3e 00 jmp 0x7c ; 0x7c
4c: 0c 94 3e 00 jmp 0x7c ; 0x7c
50: 0c 94 3e 00 jmp 0x7c ; 0x7c
54: 0c 94 3e 00 jmp 0x7c ; 0x7c
58: 0c 94 3e 00 jmp 0x7c ; 0x7c
5c: 0c 94 3e 00 jmp 0x7c ; 0x7c
60: 0c 94 3e 00 jmp 0x7c ; 0x7c
64: 0c 94 3e 00 jmp 0x7c ; 0x7c
68: 11 24 eor r1, r1
6a: 1f be out 0x3f, r1 ; 63
6c: cf ef ldi r28, 0xFF ; 255
6e: d8 e0 ldi r29, 0x08 ; 8
70: de bf out 0x3e, r29 ; 62
72: cd bf out 0x3d, r28 ; 61
74: 0e 94 40 00 call 0x80 ; 0x80
78: 0c 94 52 00 jmp 0xa4 ; 0xa4
7c: 0c 94 00 00 jmp 0 ; 0x0
80: 56 9a sbi 0x0a, 6 ; DDRD
```

```
82: 51 98 cbi 0x0a, 1 ; DDRD
84: 59 9a sbi 0x0b, 1 ; PORTD
86: 49 99 sbic 0x09, 1; PIND
88: 02 c0 rjmp +3; 0x8e
8a: 5e 9a sbi 0x0b, 6 ; PORTD
8c: 01 c0 rjmp .+2 ; 0x90
8e: 5e 9a cbi 0x0b, 6 ; PORTD
90: 22 e6 ldi r18, 0x62 ; 98
92: 82 ef ldi r24, 0xF2 ; 242
94: 90 e2 ldi r25, 0x20 ; 32
96: 21 50 subi r18, 0x01 ; 1
98: 80 40 subi r24, 0x00 ; 0
9a: 90 40 subi r25, 0x00 ; 0
9c: e1 f7 brne .-8 ; 0x96
9e: 00 c0 rjmp .+0 ; 0xa0
a0: 00 c0 rjmp .+0 ; 0xa0
a2: f1 cf rjmp .-30 ; 0x86
a4: cli
a6: ff cf rjmp .-2; 0xa6
```

Последняя команда выполняет переход на адрес 0x86, следовательно, там начало цикла.

Команды с 0x86 по 0x8e – это условный оператор.

if((PIND(PIND 1= =1)) переход на 0x88 // rjmp .+3 2 цикла
PORTD &= ~(1 <<PIND 6);
else ((PIND(PIND 1= =0)) переход на 0x8a // rjmp .+2

```
90: 22 e6 ldi r18, 0x62 ; 98
92: 82 ef ldi r24, 0xF2 ; 242
94: 90 e2 ldi r25, 0x20 ; 32
```

Команды с 0x90 по 0x94 –загрузка константных значений в регистры

Команда 9c: e1 f7 brne .-8 является условием и в случае, если Z == 0, то есть результат предыдущего значения не равен нулю, переходим по указанному адресу 0x96.

Из этого следует, что команды с 0x96 по 0x9c являются циклом.

Вычисления количества циклов:
Регистры 18, 24, 25 здесь представляют одно число N = 0x20F262 = 2 159 202,
Количество итераций = N = 2 159 202

Общее количество циклов составит 5 * N – 1 = 10 796 009

rjmp .+0 добавит 2 цикла

nop добавит 1 цикл

Загрузка в регистры ещё 3 такта

Итого: 10 796 015 циклов

При известном значении тактовой частоты = 16 МГц можно вычислить длительность задержки цикла.
t = 10 796 015 / 16000000 = 0,67475 сек или 675мс

Код на С:

```
int main(void)
{
	DDRD | = (1 << PIND6);
	DDRD &= ~ (1 << PIND1);
	PORTD | = (1 << PIND1);

	while (1)
	{
		if (PIND & (1 << PIND1))
			PORTD &= ~ (1 << PIND6);
		else
			PORTD | = (1 << PIND6);
		_delay_ms(675);
	}
}
```

2. Практическая часть

Мой ассемблерный код из первой лабораторной работы:

0: 0c 94 34 00 jmp 0x68;
4: 0c 94 3e 00 jmp 0x7c;
8: 0c 94 3e 00 jmp 0x7c;
c: 0c 94 3e 00 jmp 0x7c;
10: 0c 94 3e 00 jmp 0x7c;
14: 0c 94 3e 00 jmp 0x7c;
18: 0c 94 3e 00 jmp 0x7c;
1c: 0c 94 3e 00 jmp 0x7c;
20: 0c 94 3e 00 jmp 0x7c;
24: 0c 94 3e 00 jmp 0x7c;
28: 0c 94 3e 00 jmp 0x7c;
2c: 0c 94 3e 00 jmp 0x7c;
30: 0c 94 3e 00 jmp 0x7c;
34: 0c 94 3e 00 jmp 0x7c;
38: 0c 94 3e 00 jmp 0x7c;
3c: 0c 94 3e 00 jmp 0x7c;
40: 0c 94 3e 00 jmp 0x7c;
44: 0c 94 3e 00 jmp 0x7c;
48: 0c 94 3e 00 jmp 0x7c;
4c: 0c 94 3e 00 jmp 0x7c;
50: 0c 94 3e 00 jmp 0x7c;
54: 0c 94 3e 00 jmp 0x7c;
58: 0c 94 3e 00 jmp 0x7c;
5c: 0c 94 3e 00 jmp 0x7c;
60: 0c 94 3e 00 jmp 0x7c;
64: 0c 94 3e 00 jmp 0x7c;

68: 11 24 eor r1, r1;
6a: 1f be out SREG, r1;

6c: cf ef ser r28;

6e: d8 e0 ldi r29, 0x08;

70: de bf out SPH, r29; SPH

72: cd bf out SPL, r28; SPL

74: 0e 94 40 00 call 0x80;

78: 0c 94 50 00 jmp 0xa0;

7c: 0c 94 00 00 jmp 0;

80: 53 9a sbi DDRD, 3;

82: 52 98 cbi DDRD, 2;

84: 5a 9a sbi PORTD, 2;

86: 4a 99 sbic PIND, 2;

88: 02 c0 rjmp .+4;

8a: 5b 9a sbi PORTD, 3;

8c: 01 c0 rjmp .+2;

8e: 5b 98 cbi PORTD, 3;

90: 22 e5 ldi r18, 0x52;

92: 8e e4 ldi r24, 0x4E;

94: 97 e1 ldi r25, 0x17;

96: 21 50 subi r18, 0x01;

98: 80 40 sbci r24, 0x00

9a: 90 40 sbci r25, 0x00;

9c: e1 f7 brne .-8;

9e: 00 c0 rjmp .+0;

a0: 00 c0 rjmp .+0;

a2: f1 cf rjmp .-30;

a4: f8 94 cli ;

a6: ff cf rjmp .-2;

Анализ

Разберем его по примеру из теоретических сведений.

a2: f1 cf rjmp .-30; выполняет переход на 0x86, следовательно, там начало цикла.

Команды с 0x86 по 0x8e – это условный оператор.

```
if (PIND &(1<<PIND2))
        PORTD &=~(1<<PIND3);
else
        PORTD |=(1<<PIND3);
```

```
90: 22 e5 ldi r18, 0x52;
92: 8e e4 ldi r24, 0x4E;
94: 97 e1 ldi r25, 0x17;
```

Команды с 0x90 по 0x94 – загрузка константных значений в регистры.

```
96: 21 50 subi r18, 0x01;
98: 80 40 sbci r24, 0x00
9a: 90 40 sbci r25, 0x00;
9c: e1 f7 brne .-8;
```

Команда 9c: e1 f7 brne .-8; является условием и в случае, если Z == 0,то есть результат предыдущего значения не равен нулю, переходим по указанному адресу 0x96.

Из этого следует, что команды с 0x96 по 0x9c являются циклом.

Вычисления количества циклов:

Регистры 18, 24, 25 здесь представляют одно число N = 0x174E52 = 1 527 378,

Количество итераций = N = 1 527 378

Общее количество циклов составит 5 * N – 1 = 7 636 889

Две команды rjmp .+0 добавят 4 цикла

Загрузка в регистры ещё 3 такта

7 636 889 + 4 + 3 = 7 636 896

Итого: 7 636 896 циклов

При известном значении тактовой частоты = 16 МГц можно вычислить длительность задержки цикла.

t = 7 636 896 / 16000000 = 0,477306 сек или 477 мс

Код на С:

```
int main(void)
{
	DDRD |= (1 << PIND3);
	DDRD &= ~ (1 << PIND2);
	PORTD |= (1 << PIND2);



	while (1)
	{
		if  (PIND & (1 << PIND2))
			PORTD &= ~(1 << PIND3);
		else
			PORTD |= (1 << PIND3);
		_delay_ms(477);
	}
}
```

Выводы

Я научился переводить ассемблерный код в код на С.